Project 16.1 — CourseFit-Course-Prerequisite-Readiness-Checker

A practice twin of Project 16 — JobMatch. Same skills — semantic matching with RAG (search by meaning, not keywords) plus a deterministic score-and-gap engine — on a new real-world job. **There are no steps here on purpose.** Read the story, build it yourself, and open the hints or solution only if you get stuck.

Type: RAG pipeline (extract → retrieve → match → report) · **Stack:** Python, LangChain, Chroma (RAG), OpenAI SDK, Pydantic · **Level:** Intermediate

> **Stuck, or want to check your work?** Full hints · Complete solution

The Story — what you are going to build

A student wants to take an advanced course, but it has prerequisites — "linear algebra", "basic Python", "intro statistics". Do they actually have the background? Reading their transcript line by line to check is slow, and a keyword match misses "MATH 221: Matrices and Vectors" when the prereq just says "linear algebra".

You are going to build **CourseFit**: give it a course's prerequisites and a student's background, get back a **readiness score** and the **missing prerequisites**.

- A **prereq extractor** reads the course description and returns a typed list of required prerequisites.
- A **RAG layer** chunks the student's background (transcript, past courses, experience), embeds it, and stores it in Chroma.
- A **matcher** searches the background semantically for each prerequisite: close enough (distance below a cutoff) means they have it; otherwise it's a gap.
- A deterministic step turns met/unmet into a **readiness score** and the list of **missing prerequisites**.

The one lesson of this project: same as JobMatch — matching by *meaning* is a RAG problem. "MATH 221: Matrices and Vectors" should satisfy a "linear algebra" prerequisite, and only a semantic search can see that. The score-and-gap engine stays deterministic (a distance below a cutoff is a match), so you can test the whole decision without a model.

How it works, from two files to a readiness report (picture the whole machine)

A course description and a student background come in. The **prereq extractor** pulls the required prerequisites into a typed list. The **RAG layer** chunks and embeds the background into Chroma. The **matcher** takes each prerequisite and searches the background: if the closest chunk is near enough, it's **met**; if not, it's **missing**. A deterministic step computes the **score** (percent met) and the **missing** list, and a formatter prints the report.

That's the whole machine: **course + background → extract prereqs + index background → match each by distance → readiness score + missing prereqs.**

What you are building (the pieces)

- Typed shapes: `RequiredPrereqs(prereqs)`, `ReadinessReport(score, met, missing)`.
- A deterministic `check_prereqs(required, distance, cutoff)`.
- A RAG layer over the background (chunk + embed + `best_distance`).
- A prereq extractor (`with_structured_output`) and a pipeline (`run_check`).

Inputs: a course description + a student background. **Outputs:** a readiness score and the missing prerequisites. **Rules:** match by meaning (RAG); the score engine takes a `distance` function so it's testable with no model; no prerequisites means 100% ready (not a crash).

A real example to build and test against

- A background covering calculus, Python, and matrices, against a course needing "linear algebra, Python, statistics" → met: linear algebra, Python; missing: statistics; a readiness score.
- A background with nothing relevant → 0% ready, everything missing (and the engine still returns a valid report).

How to practice this

1. Read this story. **Don't open the solution yet.** (Review Project 16 / Doc08 RAG if hazy.)
2. Build the score-and-gap engine **first** and test it with a **fake** `distance(prereq)` — no model, no embeddings.
3. Build the RAG layer (chunk + embed + `best_distance`) and the prereq extractor.
4. Wire them: for each prereq, `best_distance(collection, prereq)`; below the cutoff is met.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

Why this project is useful (read this at the end)

"Do they meet the requirements, and if not, what's missing?" is a job everywhere — course prereqs here, but the same shape fits onboarding readiness, certification checks, loan eligibility, and feature-gap analysis. The skill is the structure: extract the requirements, make the candidate's evidence searchable by meaning, match each requirement by distance, and report the gaps. Build CourseFit and you can build any semantic requirements-checker.

Done when

- Matching is semantic (RAG), so "Matrices and Vectors" meets a "linear algebra" prereq.
- `check_prereqs` takes a `distance` function and is tested with no model.
- The score is a percentage, and zero prerequisites doesn't divide by zero.
- The report names the exact missing prerequisites.
- Your tests pass: the engine (fake distances) and a live check on a real background.

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.